Reinforcement Learning Course: Week 3 Summary

Policy-Based Methods, Actor-Critic Architectures, and PPO

1 Introduction: The Limitations of Value-Based Methods

Throughout Weeks 1 and 2, we explored value-based methods such as Q-Learning and Deep Q-Networks (DQN). These algorithms operate by estimating action-values (Q-values) and subsequently deriving policies via greedy selection (typically using the max operator).

However, value-based approaches face severe limitations in two major scenarios:

1. **Continuous Action Spaces:** When actions span a continuous spectrum (e.g., steering angles or motor torques on a continuous scale like $[-1, 1]$), computing $\max_a Q(s, a)$ requires solving a complex optimization problem at every single time step.
2. **Stochastic Policies:** In environments with partial observability or adversarial settings (like poker or rock-paper-scissors), the optimal strategy requires a probability distribution over actions rather than a single deterministic choice. Value-based methods cannot directly represent these smoothly.

To overcome these barriers, Week 3 introduces **Policy-Based Methods**, where neural networks parameterize policies directly, laying the groundwork for hybrid architectures like Actor-Critic and modern optimization algorithms.

2 Policy-Based Methods and the REINFORCE Algorithm

Instead of approximating action-values to infer actions, a Policy Network parameterized by weights θ takes the environment state s as input and outputs a direct probability distribution over all available actions:

$$\pi_\theta(a|s) = P(A_t = a|S_t = s; \theta)$$

2.1 The Policy Gradient Theorem

Our goal is to optimize network weights θ to maximize an objective function $J(\theta)$ representing expected cumulative rewards. The Policy Gradient Theorem allows us to compute the gradient of this objective without requiring explicit environmental transition probabilities:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) G_t\right]$$

Intuitive Meaning: This expression measures how parameters affect action probabilities. If a trajectory yields a high cumulative return ($G_t > 0$), weights are adjusted to increase the likelihood of those actions. Conversely, negative returns suppress them.

2.2 The REINFORCE Algorithm

REINFORCE is a Monte Carlo policy gradient method. Because it relies on complete episodes to compute actual returns G_t, the agent must execute a full episode before performing a weight update:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a|s) G_t$$

Major Drawback: Waiting until the end of an episode introduces high variance in cumulative returns, making training slow and unstable in complex environments.

3 Hybrid Methods: Actor-Critic Architecture

To address the high variance of pure policy methods, Actor-Critic architectures merge value-based and policy-based principles into two cooperative components:

- **The Actor:** Maintains the policy parameterization $\pi_\theta(a|s)$, selects actions, and updates its parameters based on feedback from the critic.
- **The Critic:** Evaluates states by learning a state-value function $V_\phi(s)$, providing fine-grained numerical feedback to guide the actor.

3.1 The Advantage Function

Instead of relying on raw, noisy returns G_t, the critic calculates the **Advantage Function** $A(s, a)$, quantifying how much better a specific action performs compared to the expected average value of that state:

$$A(s, a) = Q(s, a) - V(s) \approx R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Using Temporal Difference (TD) error as an advantage estimate dramatically cuts down variance, stabilizing gradient updates.

4 Proximal Policy Optimization (PPO)

Despite improvements from Actor-Critic models, traditional policy gradients suffer from a critical vulnerability: **Policy Collapse**. A single overly aggressive gradient update caused by an anomalous batch can catastrophically destroy a well-tuned policy, rendering the agent completely dysfunctional.

Proximal Policy Optimization (PPO) prevents this by bounding policy updates within a safe trust region.

4.1 1. Probability Ratio

PPO introduces a probability ratio $r_t(\theta)$ comparing action probabilities under the current policy versus the old policy (the policy used to collect the experience):

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

4.2 2. The Clipped Surrogate Objective

To restrain destructive updates, PPO clips the probability ratio inside a controlled interval $[1 - \epsilon, 1 + \epsilon]$ (typically $\epsilon = 0.2$, restricting changes to $\pm 20\%$):

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t \right) \right]$$

Mechanism of the Clipping Function:

- When advantages are positive ($\hat{A}_t > 0$), the action yielded good outcomes, so we want to increase its probability. However, if the ratio exceeds $1 + \epsilon$, the min operator caps the objective, neutralizing further upward pressure.
- When advantages are negative ($\hat{A}_t < 0$), clipping similarly restricts harmful downward drops.

4.3 3. Batching and Multi-Epoch Training (K-Epochs)

Unlike online methods that discard trajectories immediately after a single update step, PPO gathers a large experience batch (e.g., 2000 steps) and safely reuses it across multiple mini-batch optimization epochs (K epochs) under the protection of the clipping constraint, maximizing sample efficiency.